

POC Test Cases — Data Platform Validation



Huawei Technologies Co., Ltd.

v2.3.1 — September 27, 2026 16:55

Contents

1	Introduction	2
1.1	Test Scope	2
1.2	Preconditions and Preparations.....	2
1.3	Acceptance Method.....	3
2	Test Cases	4
2.1	Platform Architecture	4
2.2	Data Engineering.....	7
2.3	Security and Access Control	10
2.4	Environment Setup and Tooling	12
3	Conclusion	16
3.1	Test Architecture Diagrams.....	16
3.2	Brand Color Palette	18
4	Changelog.....	19

Info

This document demonstrates the `testbook.cls` template for Huawei Cloud POC/acceptance test case documents. It exercises all available commands and environments: cover page, table of contents, test case environment with all field commands, mini header bars, testprocedure environment with step-by-step actions, objectives block, code blocks, images, callouts (`warning`, `tip`, `infobox`), tables, badge, menu paths, weblinks, notes, param references, test summary table, result badges, and changelog.

1

Introduction

New

General Objective: Validate the Huawei Cloud Data Platform deployment for the POC environment, ensuring that all core services (DataArts Studio, MRS, OBS, VPC) are provisioned correctly, accessible, and functioning as specified in the solution architecture.

1.1 Test Scope

Domain	What Should Be Observed
Platform Architecture	Core services are deployed, healthy, and reachable via the Console API.
Data Engineering	DataArts Studio pipelines can ingest, transform, and load data into the target storage.
Data Storage	OBS buckets and MRS HDFS accept writes/reads with correct permissions.
Security	IAM policies, VPC isolation, and security group rules enforce the intended access control.

Table 1: Test scope by domain.

1.2 Preconditions and Preparations

- All infrastructure resources must be provisioned via Terraform and the `terraform apply` output must show zero errors.
- The MRS cluster must be in **Running** state before any data engineering test case is executed.
- Test data files (CSV/JSON) must be uploaded to the designated OBS bucket prior to pipeline execution.
- Each tester must have a personal IAM account with the **Dayu_User** role assigned in DataArts Studio.
- Network firewall rules must allow inbound TCP 22 (SSH) and TCP 443 (HTTPS) from the test execution subnet.

Note: Ensure the test execution environment has the correct timezone set to *America/Sao_Paulo* (GMT-3) so that log timestamps align with the Huawei Cloud Console.

```
# Verify CLI tools are installed
```

```
terraform version
obsutil version
kubectl version --client
```

Tip

Use `terraform plan` before `terraform apply` to preview infrastructure changes and avoid unintended resource modifications.

1.3 Acceptance Method

Result	Explanation
Pass	The test case passes fully — observed behavior matches the expected result with no deviations.
Partial	The test case was executed successfully, but with restrictions, dependencies, or additional configuration (documented in Remarks).
Fail	The test case fails — observed behavior does not match the expected result. A defect report must be filed.
Untested	The test case was not executed — execution remains pending and must be scheduled.

Table 2: Acceptance criteria.

2.1 Platform Architecture

Testcase 1: Verify MRS cluster deployment and health

Objective

Confirm that the MapReduce Service (MRS) cluster is deployed with the required components (HDFS, YARN, Spark, Hive) and all nodes report a healthy status.

Test Scope

MRS cluster deployment; Cluster health monitoring; Node status verification

Prerequisites

1. Terraform infrastructure apply completed successfully.
2. MRS cluster ID is available in the Terraform output.
3. IAM user has **MRS_Viewer** or higher role.

Procedure

1. Log in to the Huawei Cloud Console
2. Navigate to **Console** → **MapReduce Service** → **Clusters**
3. Locate cluster `mrs-poc-cluster`
4. Verify cluster status
5. Review Component tab
6. Review Node tab



Expected Result

1. Cluster status is **Running**.

2. All four components (HDFS, YARN, Spark, Hive) are listed with status **Normal**.
3. All nodes report status **Running** with no alarms.

Test Result

Pass

Remarks

If any component shows **Abnormal**, check the MRS alarm page before marking the result.

Testcase 2: Verify OBS bucket creation and accessibility

Objective

Confirm that the Object Storage Service (OBS) bucket used for the POC data lake is created, has the correct storage class, and is accessible with the designated IAM policy.

Test Scope

OBS bucket creation; Data accessibility; Storage configuration

Prerequisites

1. OBS bucket name is defined in the Terraform configuration.
2. IAM policy granting read/write access to the bucket is attached to the test user.

Procedure

1. Log in to the Console
2. Navigate to **Console** → **Object Storage Service**
3. Locate the bucket (e.g. poc-data-lake-raw)
4. Verify the storage class is **Standard**
5. Upload a test file (test-upload.txt) to the bucket
6. Download the file and compare its content with the original
7. Delete the test file

Expected Result

1. Bucket exists with storage class **Standard**.
2. File upload completes without error.

3. Downloaded file content matches the original.

4. File deletion succeeds.

Test Result

Pass

Remarks

OBS eventual consistency may cause a brief delay before a newly uploaded object appears in listings.

Testcase 3: Verify VPC and security group configuration

Objective

Confirm that the VPC, subnet, and security groups are configured according to the network architecture diagram, with the correct CIDR blocks and inbound/outbound rules.

Test Scope

VPC configuration; Subnet setup; Security group rules

Prerequisites

1. VPC and subnet IDs are available from the Terraform output.
2. Security group names are documented in the solution architecture.

Procedure

1. Log in to the Console
2. Navigate to **Console** → **Virtual Private Cloud** → **VPCs**
3. Locate the VPC (e.g. vpc-poc) and verify its CIDR block matches the architecture (e.g. 10.0.0.0/16)
4. Click the subnet and verify its CIDR (e.g. 10.0.1.0/24)
5. Navigate to **Security Groups** and verify inbound rules: TCP 22 from the bastion subnet, TCP 443 from the corporate proxy
6. Verify outbound rules allow all traffic (default)

Expected Result

1. VPC CIDR is 10.0.0.0/16.
2. Subnet CIDR is 10.0.1.0/24.

3. Inbound rules match the architecture: TCP 22 and TCP 443 from the specified sources.
4. Outbound allows all traffic.

Test Result

Pass

Remarks

If the security group has additional rules beyond the architecture spec, document them in the Remarks field.

2.2 Data Engineering

Testcase 4: Verify DataArts Studio workspace creation

Objective

Confirm that the DataArts Studio (Dayu) instance is provisioned, the workspace is accessible, and the designated IAM users can log in with the **Dayu_User** role.

Test Scope

DataArts Studio provisioning; Workspace access; IAM role assignment

Prerequisites

1. DataArts Studio instance is provisioned and its status is **Running**.
2. Test IAM user has the **Dayu_User** role assigned.

Procedure

1. Log in to the Console with the test IAM user
2. Navigate to **Console** → **DataArts Studio** → **Workspaces**
3. Locate the workspace (e.g. poc-workspace)
4. Click **Access Workspace** to open the DataArts Studio console
5. Verify the left navigation panel loads with the expected modules: **Data Integration**, **Data Development**, **Data Architecture**

Expected Result

1. Workspace is listed and accessible.

2. DataArts Studio console opens without errors.
3. All three modules (Data Integration, Data Development, Data Architecture) are visible in the navigation.

Test Result

Pass

Remarks

If the workspace fails to load, check the Dayu instance status and the IAM role assignment.

Testcase 5: Verify data ingestion pipeline execution

Objective

Confirm that a DataArts Studio batch pipeline can successfully ingest CSV data from OBS, apply a basic transformation, and write the result to the target OBS path.

Test Scope

Data ingestion pipeline; OBS-to-OBS transformation; Data pipeline execution

Prerequisites

1. Test Case 4 completed (DataArts Studio workspace accessible).
2. Source CSV file (`customers.csv`) exists in the OBS raw bucket.
3. Target OBS path (`poc-datalake-curated/customers/`) is configured.
4. Pipeline `pl-ingest-customers` is published in DataArts Studio.

Procedure

1. Navigate to Data Development in DataArts Studio
2. Locate pipeline `pl-ingest-customers`
3. Click Run to execute the pipeline
4. Wait for pipeline status (timeout: 10 min)
5. Verify output file in OBS
6. Download and verify row count

```
# Verify the output file in OBS via CLI
obsutil ls obs://poc-datalake-curated/customers/ -limit=10
obsutil stat obs://poc-datalake-curated/customers/output.csv
```

Expected Result

1. Pipeline execution completes with status **Success**.
2. Output file is created in the target OBS path.
3. Row count of the output file matches the source file.

Test Result

Pass

Remarks

Pipeline execution time varies with data volume. For the POC, the source file contains approximately 10,000 rows.

Testcase 6: Verify Spark SQL query on MRS cluster

Objective

Confirm that a Spark SQL query can be executed on the MRS cluster to read data from a Hive table and return correct results, validating the integration between MRS and the data lake.

Test Scope

Spark SQL execution; Hive table integration; MRS data lake validation

Prerequisites

1. Test Case 1 completed (MRS cluster healthy).
2. Hive table `poc_db.customers` exists and contains data loaded by the ingestion pipeline.
3. SSH access to the MRS master node is configured.

Procedure

1. SSH into the MRS master node
2. Run the Spark SQL CLI:

```
spark-sql --master yarn \  
  --conf spark.sql.hive.convertMetastoreOrc=true \  
  -e "SELECT COUNT(*) FROM poc_db.customers;"
```

3. Verify the returned count matches the expected row count (10,000)
4. Run a sample query to verify data integrity:

```
spark-sql --master yarn \  
-e "SELECT customer_id, name FROM poc_db.customers LIMIT 5;"
```

5. Confirm the query returns 5 rows with non-null values

Expected Result

1. COUNT(*) returns 10,000.
2. The sample query returns 5 rows with valid customer_id and name values.

Test Result

Pass

Remarks

If the Spark SQL CLI is not available, use the MRS Spark2x component's web UI to submit the query instead.

2.3 Security and Access Control

Testcase 7: Verify IAM policy enforcement on OBS buckets

Objective

Confirm that IAM policies correctly restrict OBS bucket access — authorized users can read/write while unauthorized users are denied, and the bucket policy matches the principle of least privilege.

Test Scope

IAM policy enforcement; OBS access control; Least privilege validation

Prerequisites

1. OBS bucket poc-data-lake-raw exists with the designated IAM policy.
2. Two test IAM users: one with **OBS_ReadWrite** role, one with no OBS permissions.
3. obsutil CLI configured for both users.

Procedure

1. Using the authorized user, upload a test file to poc-data-lake-raw

2. Using the authorized user, download and verify the file content
3. Using the unauthorized user, attempt to upload a file — verify the request is denied (HTTP 403)
4. Using the unauthorized user, attempt to list objects in the bucket — verify the request is denied
5. Review the bucket policy in the Console at **Console** → **Object Storage Service** → **Bucket Policies** and confirm it grants only the intended permissions

```
# Authorized user – upload and verify
obsutil cp test-file.txt obs://poc-datalake-raw/test-file.txt
obsutil ls obs://poc-datalake-raw/ -limit=5

# Unauthorized user – expect 403 errors
obsutil cp test-file.txt obs://poc-datalake-raw/unauthorized.txt
obsutil ls obs://poc-datalake-raw/
```

Expected Result

1. Authorized user can upload and download successfully.
2. Unauthorized user receives HTTP 403 on both upload and list operations.
3. Bucket policy grants only the intended read/write permissions to the designated roles.

Test Result

Fail

Remarks

Check the Cloud Trail logs if access denial is not returned as expected — there may be a policy inheritance from the project or domain level.

Note: IAM policy changes can take up to 5 minutes to propagate. Wait before re-testing after modifying a policy.

Testcase 8: Verify DataArts Studio data masking rule

Objective

Confirm that sensitive data columns (e.g. PII fields) are masked according to the DataArts Studio data masking rule when queried by non-admin users.

Test Scope

Data masking rules; PII protection; Non-admin data access

Prerequisites

1. Test Case 4 completed (DataArts Studio workspace accessible).
2. Data masking rule configured for column `customers.ssn` (Social Security Number).
3. Test user has **Dayu_User** role (not admin).

Procedure

1. In the DataArts Studio console, navigate to **Data Architecture** → **Data Masking**
2. Verify the masking rule for `customers.ssn` is active and uses the **Mask All** strategy
3. Query the `customers` table as the test user via DataArts Studio SQL console
4. Verify the `ssn` column returns masked values (e.g. `*****`) instead of real data
5. Query the same table as an admin user and verify real values are returned

Expected Result

1. Masking rule is active for `customers.ssn`.
2. Non-admin user sees masked values in the `ssn` column.
3. Admin user sees real values in the `ssn` column.

Test Result

Partial

Remarks

Result with caveats: data masking rules apply at query time — the underlying data in OBS/HDFS is not modified, so masking does not cover direct bucket exports.

2.4 Environment Setup and Tooling

Testcase 9: Verify Terraform infrastructure deployment

Objective

Confirm that the Terraform configuration deploys all required infrastructure resources (VPC, MRS, OBS, DataArts Studio) without errors and the state file reflects the expected resources.

Test Scope

Infrastructure as Code; Terraform deployment; Resource state verification

Prerequisites

1. terraform CLI installed (version $\geq$ 1.0).
2. AK/SK credentials configured via environment variables or *provider.tf*.
3. Terraform configuration files in the project directory.

Procedure

1. Navigate to the Terraform project directory
2. Run `terraform init` to initialize the working directory
3. Run `terraform plan` and review the planned changes — confirm all expected resources are listed
4. Run `terraform apply -auto-approve` and wait for completion
5. Run `terraform output` and verify all output values are populated
6. Run `terraform state list` and confirm the resource count matches the architecture

```
# Initialize and deploy infrastructure
cd /opt/poc-infra/terraform
terraform init
terraform plan -out=tfplan
terraform apply tfplan

# Verify deployment
terraform output
terraform state list | wc -l
```

Expected Result

1. `terraform apply` completes with no errors.
2. All output values are populated (cluster ID, bucket name, VPC ID).
3. Resource count in state matches the architecture specification.

Test Result

Pass

Remarks

If terraform apply fails, check the AK/SK credentials and the sa-brazil-1 region quota limits.

Important

Do not skip the terraform plan review step. Applying unreviewed changes to production-adjacent environments can cause data loss or service disruption.

Testcase 10: Verify test data upload to OBS

Objective

Confirm that all required test data files (CSV, JSON) can be uploaded to the designated OBS bucket and are accessible by the DataArts Studio pipeline.

Test Scope

OBS data upload; Test data preparation; Data pipeline connectivity

Prerequisites

1. Test Case 2 completed (OBS bucket accessible).
2. Test data files prepared in the local directory `./test-data/`.
3. obsutil CLI configured with the test user credentials.

Procedure

1. List the local test data files and verify their integrity (row count, schema)
2. Upload all files to the OBS raw bucket using obsutil
3. Verify each file exists in OBS by listing the bucket contents
4. Download a sample file from OBS and compare it with the local original (checksum)
5. In DataArts Studio, verify the data source connection can read the uploaded files

```
# Upload test data files to OBS
obsutil cp ./test-data/customers.csv
          obs://poc-datalake-raw/customers.csv
obsutil cp ./test-data/transactions.json
          obs://poc-datalake-raw/transactions.json
```

```
# Verify uploads
obsutil ls obs://poc-datalake-raw/ -limit=20
```

Expected Result

1. All test data files are uploaded without errors.
2. Bucket listing shows all uploaded files with correct sizes.
3. Downloaded file checksum matches the local original.
4. DataArts Studio data source connection can read the files.

Test Result

Untested

Remarks

Large files (>1 GB) should be uploaded using the OBS multipart upload feature for better reliability.

Note: Refer to the [Huawei Cloud OBS Documentation](#) for advanced upload options and troubleshooting.

[Image placeholder]

File: assets/test-execution-dashboard.png

Description: Screenshot of the test execution dashboard showing all test case results

Place the image file at the path above, then replace this command with

*\image{assets/test-execution-dashboard.png} or
\imagecap{assets/test-execution-dashboard.png}{caption}.*

ID	Title	Status
1	Verify MRS cluster deployment and health	Pass
2	Verify OBS bucket creation and accessibility	Pass
3	Verify VPC and security group configuration	Pass
4	Verify DataArts Studio workspace creation	Pass
5	Verify data ingestion pipeline execution	Pass
6	Verify Spark SQL query on MRS cluster	Pass
7	Verify IAM policy enforcement on OBS buckets	Fail
8	Verify DataArts Studio data masking rule	Partial
9	Verify Terraform infrastructure deployment	Pass
10	Verify test data upload to OBS	Untested

3.1 Test Architecture Diagrams

Info

This section and the following (“Brand Color Palette”) are sample demonstration content — they are not part of a real test document.

Diagrams can be used to visualize test flows and architectures.

**Diagram 1:** Sequence diagram of test execution

**Diagram 2:** Test flowchart

3.2 Brand Color Palette

The template defines auxiliary brand colors aligned to the Huawei Cloud Brand Guidelines:

Color	HEX	Pantone	Sample
Rose Red	#C40054	7636	
Dark Red	#7F0001	483C	
Orange	#ED6D00	165C	
Yellow	#FCC800	7406C	
Green	#62B230	3501C	
Blue	#30B5C5	2227C	

4

Changelog

2.3.1 2026-09-28

- Added note indicating that the diagrams and color palette sections are sample demonstration content.

2.3.0 2026-09-28

- Removed duplicate prerequisites from the objectives block — only the general objective now appears in the block; prerequisites remain in the dedicated “Preconditions and Preparations” section.
- Fixed cover text casing: “CO., LTD” → “Co., Ltd.”

2.2.0 2026-09-23

- Unified result vocabulary: the acceptance criteria table now uses Pass, Partial, Fail, and Untested.
- Replaced the Blocked result with Partial: Test Case 8 and its test summary row were converted, with remarks aligned to the partial result.

2.1.1 2026-09-19

- Fixed changelog dates — replaced \today with the actual historical release dates.
- Fixed malformed row in the test-scope table (Security domain) — the first cell separator was missing.

2.1.0 2026-09-13

- Restructured document to 3-section layout: Introduction, Test Cases (with subsections per domain), and Conclusion (with test summary table).
- Removed manual TC-XXX prefixes from testcase titles — the environment auto-numbers them as “Test Case 1:”, “Test Case 2:”, etc.
- Updated testsummary IDs from TC-XXX to auto-numbered 1–10.
- Changed teststeps from tabular table to auto-numbered paragraphs. \teststep now takes 1 arg (action only). Images, code blocks, and callouts can be placed freely between steps.

2.0.0 2026-09-13

- Redesigned testcase layout: full-width mini header bars for each field, testprocedure environment with 2-column step table, field order updated (Test Result before Remarks).

1.1.0 2026-09-12

- Added comprehensive feature demonstrations: warning, tip, infobox callouts; code blocks; badge; weblink; note; param; image, imagecap, and imageplaceholder; menu paths; and inlinecode references.
- Added Security and Access Control test cases.
- Added Environment Setup and Tooling test cases.



1.0.0

2026-09-12

- Initial version.
 - Added Platform Architecture test cases.
 - Added Data Engineering test cases.
-